เตรียมตอบ Demo: Mini Job Scheduler (517-312)

ร่างโดย Claude เพื่อใช้อ่านทบทวน ตัวเลขทุกตัวมาจากการรันจริงของคุณ ก่อน Demo ให้ลองเล่าแต่ละหัวข้อด้วยคำของตัวเองโดยไม่ดูเอกสาร ตรงไหนเล่าไม่ได้ แปลว่าตรงนั้นยังต้องทำความเข้าใจเพิ่ม

ส่วนที่ 1: 8 หัวข้อที่อาจารย์บอกว่าจะถาม

1. เปลี่ยนจำนวน Worker หรือ permit แล้วระบบเปลี่ยนอย่างไร

รัน (Priority)	Throughput	avg Resource Wait
standard, Worker 1	0.41	0
standard, Worker 3	1.08	254
standard, Worker 5	1.30	711
printer, Worker 3, Printer 1	0.80	1618
printer, Worker 3, Printer 2	1.43	306

- **Worker 1:** ทำทีละงาน ไม่มีใครแย่ง resource จึงรอ resource = 0 แต่ช้าที่สุด
- **Worker 3 เป็น 5:** Throughput เพิ่มแค่ราว 20% (ไม่ใช่ 67% ตามสัดส่วน) เพราะ PRINTER มีแค่ 1 เครื่อง Worker ที่เพิ่มมาก็มาแย่งรอที่ PRINTER รอ resource จึงเพิ่มจาก 254 เป็น 711
- **Printer 1 เป็น 2:** รอ resource ลดมาก Throughput เพิ่มเกือบ 1.8 เท่า คอขวดย้ายบางส่วนไปที่จำนวน Worker (3 ตัว)
- **วิธีคาดคะเนเร็วๆ:** เวลาที่ต้องถือ resource รวม หารด้วยจำนวน permit คือขอบล่างของเวลารวม เช่น printer 6 งาน x 1200 = 7200 ms หาร 1 = 7200 (จริง 7506) หาร 2 = 3600 (จริง 4204 เพราะมีช่วงรอ Worker ว่างและช่วงทำงานหลักก่อนขอเครื่อง)

2. ข้อมูลร่วมที่หลาย Thread แก้ไข และถ้าเอาการป้องกันออก

ข้อมูลร่วม	ใครแก้ / ใครอ่าน	ป้องกันด้วย	ถ้าเอาออก
PriorityQueue ใน ReadyQueue	Scheduler เพิ่ม, Worker หยิบ, Monitor อ่านขนาด	`synchronized` ทุกเมธอด	โครงสร้างภายในพัง งานหายหรือซ้ำ ลำดับ priority ผิดโดยไม่มี error และ `wait/notifyAll` จะโยน IllegalMonitorStateException (เคยเจอจริงตอนลืม `synchronized` ที่ `add`)
`running`, `completed`	Worker หลายตัวแก้, Monitor และ main อ่าน	`AtomicInteger`	lost update: Worker สองตัวอ่านค่า 5 แล้วต่างเขียน 6 นับขาดไป 1 ทำให้ completed ไม่เท่ากับจำนวนงานที่ทำจริง
ฟิลด์เวลาใน Job	Thread หนึ่งเขียน อีก Thread อ่าน	`volatile` (แต่ละฟิลด์มีผู้เขียนคนเดียว) และ `join()`	อ่านได้ค่าเก่า (-1) ทำให้ WT/TAT ผิด
`closed` ใน ReadyQueue	Scheduler เขียนใน `close()`, Worker อ่านใน `take()`	อยู่ใต้ `synchronized` ของ object เดียวกัน	Worker อาจไม่เห็น closed=true นอนรอตลอด JVM ไม่ปิด
Semaphore, `arrivalQueue`	หลาย Thread	ป้องกันอยู่ในตัวคลาสของ Java เอง	-

ทำไม `closed` ไม่ต้องเป็น `volatile`: เพราะทั้งการเขียนและการอ่านอยู่ในเมธอด `synchronized` ที่ล็อก object เดียวกัน การล็อกจึงรับประกันทั้งความเป็นก้อนเดียวและการเห็นค่าล่าสุดอยู่แล้ว หลักคือข้อมูลร่วมตัวหนึ่งต้องใช้กุญแจเดียวกันทุกครั้งที่เข้าถึง

3. FCFS / Priority และ tie-break จากโค้ดจริง

```
// PRIORITY: priority (1 มาก่อน) -> arrivalMs -> sequence
comparingInt(priority).thenComparingLong(arrivalMs).thenComparingInt(sequence
)
// FCFS: arrivalMs -> sequence
comparingLong(arrivalMs).thenComparingInt(sequence)
```

- `PriorityQueue` หยิบตัวที่ "น้อยที่สุดตาม comparator" ออกก่อนเสมอ

- **tie-break ใช้ข้อมูลของ Job เอง** (เวลาเข้า แล้วตามด้วย `sequence` คือลำดับบรรทัดในไฟล์) ไม่ขึ้นกับว่า Thread ไหนมาก่อน
- **หลักฐานจาก log:** `jobs_same_priority` เริ่มทำตามลำดับ S01 ถึง S06 ตรงกับเวลาเข้า ส่วน `jobs_db` งาน D05 (priority 1) เริ่มก่อน D04 (priority 2) ทั้งที่ D04 มาก่อน
- **non-preemptive:** J01 ถึง J03 ไม่ถูกดึงกลับเมื่อ J07 (priority 1) มาถึง งานที่ Worker รับไปแล้วทำจนจบ
- **สิ่งที่ไม่แน่นอน:** Worker ตัวไหนได้งานไหน (เช่น S01 อยู่ worker-2 ส่วน S02 อยู่ worker-1) ลำดับของ **Job** ที่ถูกเลือกไม่เปลี่ยน
- **ทำไม Priority มีค่า WT ดีกว่าแต่ Throughput ต่ำกว่า FCFS:** WT เฉลี่ย 2321 vs 2958 แต่ Throughput 1.08 vs 1.15 เพราะ Priority ดึงงานสั้นขึ้นก่อน งานยาว (J04 ถึง J06) ถูกเลื่อนไปเริ่มช้า ตอนท้ายจึงมี Worker ว่างเปล่า เวลารวมยาวขึ้น (9221 vs 8717 ms)

4. วิธีปิดระบบ และถ้าเพิ่ม Worker ระหว่างทำงาน

1. JobGenerator ปล่อยงานครบ แล้วใส่ `POISON_PILL` (งานปลอมเป็นสัญญาณ) ลง `arrivalQueue` แล้วจบ
2. Scheduler เจอ pill (เทียบด้วย `==` เพราะต้องเป็นออบเจกต์ตัวเดียวกัน) ออกจากลูป และใน `finally` เรียก `readyQueue.close()`
3. `close()` ตั้ง closed=true แล้ว `notifyAll()` ปลุก Worker ที่นอนรอ
4. `take()` ทำงานตามนี้: มีงานก็หยิบให้ตามปกติ, คิวว่างและปิดแล้วคืน `null`, คิวว่างแต่ยังไม่ปิดก็รอต่อ Worker จึงทำงานที่เหลือจนหมดก่อนแล้วค่อยออกจากลูป
5. main `join()` generator, scheduler, ทุก Worker แล้วสั่ง `monitor.interrupt()` และ `join()` ก่อนพิมพ์สรุป

ถ้าเพิ่ม Worker ระหว่างทำงาน: ยังใช้ได้ เพราะสัญญาณหยุดของเราคือ **สถานะของคิว** (ปิดแล้วและว่าง) ไม่ใช่การนับจำนวน Worker Worker ใหม่ที่เรียก `take()` บนคิวเดียวกันจะได้งานหรือได้ `null` แล้วออกเองได้ถูกต้อง ถ้าเราใช้วิธีส่ง pill ให้ครบทีละ Worker วิธีนั้นจะพัง เพราะ Worker ใหม่ไม่มี pill เป็นของตัวเอง **ข้อควรระวัง:** ต้องเพิ่ม Thread ใหม่เข้ารายการที่ main จะ `join()` ด้วย ไม่งั้น main อาจพิมพ์สรุปก่อนที่ Worker ตัวนั้นจะเสร็จ (ในโค้ดตอนนี้ array ของ Worker กำหนดขนาดตายตัวตอนเริ่ม)

5. ถ้า Worker ถูก interrupt หลัง acquire จะกัน permit หายอย่างไร

```
resources.acquire(job.resource);   // อยู่นอก try
try {
    ... Thread.sleep(job.resourceMs);
} finally {
    logger.resourceReleased(job);
    resources.release(job.resource);   // คืนเสมอ
}
```

- **interrupt หลัง acquire สำเร็จ:** โค้ดกระโดดเข้า `finally` คืน permit เสมอ
- **ทำไม `acquire` ต้องอยู่นอก `try`:** ถ้า interrupt ตอนยังรออยู่ (ยังไม่ได้ permit) แล้ว `finally` สั่ง `release()` ตาม permit จะเกินจริง (Semaphore ไม่เช็กว่าเคยได้มาก่อน) จะมี 2 งานใช้ PRINTER พร้อมกันได้โดยไม่มี error
- **ผลทดสอบจริง (`TestInterrupt`):** เคส interrupt ตอนรอ ระหว่างที่ main ยังถือ permit เห็น `printer=1/1` (ไม่คืนสิ่งที่ไม่เคยได้) และหลังคืนเป็น `0/1` ส่วนเคส interrupt ตอนถือ งานคืน permit ที่ 301 ms ทั้งที่ตั้ง `resourceMs` ไว้ 5000 และจบที่ `0/1`

6. Thread.State ตอนรอ Semaphore เทียบกับตอน sleep

ตอน	State	เหตุผล
`Semaphore.acquire()` รอ permit	`WAITING`	รอแบบไม่มีเวลากำหนด ต้องมีคน `release()` มาปลุก
`Thread.sleep(resourceMs)`	`TIMED_WAITING`	รอแบบมีเวลากำกับ หมดเวลาแล้วตื่นเอง
Worker รองานใน `ReadyQueue.take()` (`wait()`)	`WAITING`	รอให้ `notifyAll()` ปลุก
พยายามเข้าบล็อก `synchronized` ที่คนอื่นถือ	`BLOCKED`	รอกุญแจ (monitor lock)

เห็นจริงใน `TestInterrupt` : เคสรอ permit ได้ `WAITING` เคสถือ permit ได้ `TIMED_WAITING`

7. ใช้สมการ TAT = WT + workMs + Resource Wait + resourceMs หาจุดผิด

โปรแกรมพิมพ์คอลัมน์ `diff` = TAT ลบด้วยฝั่งขวาของสมการให้ทุก Job ค่าปกติคือ 2 ถึง 9 ms (งานแรกที่ใช้ resource อาจถึง ~18 ms เป็นไปได้ว่าเพราะ JVM โหลดคลาสครั้งแรก ซึ่งเป็นการเดา ยังไม่ได้พิสูจน์)

ตัวอย่างค่าปกติ D03: WT 1 + work 250 + RW 1061 + resource 1200 = 2512 เทียบกับ TAT 2516 ต่าง 4 ms

ถ้า `diff` ใหญ่ ให้ดูว่าใกล้เคียงค่าอะไร:

- ใกล้เคียง Resource Wait ของงานนั้น: น่าจะลืมบันทึก `resourceWaitMs` (ตัวอย่าง D03 ถ้าเป็น 0 diff จะเป็น ~1065)
- ใกล้เคียง workMs หรือ resourceMs: จับ `startTimeMs` หรือ `completionTimeMs` ผิดจุด
- เป็นลบ: บันทึกเวลาเริ่มช้าเกินไป หรือ WT ถูกคำนวณจากเวลาที่ไม่ใช่เวลาเข้าจริง (`actualArrivalMs`)

8. Generative AI ช่วยส่วนใด และตรวจสอบอย่างไร

ส่วนนี้ให้แก้ให้ตรงกับที่คุณทำจริงและเขียนด้วยคำของตัวเอง ข้อมูลข้างล่างคือสิ่งที่เกิดขึ้นในแชตนี้

- **AI ช่วย:** อธิบายแนวคิด (BlockingQueue, Semaphore, synchronized, poison pill) และร่างโค้ดทีละชิ้น ได้แก่ JobGenerator, Scheduler, ReadyQueue, Worker, ResourceManager, Statistics, Monitor และตัวทดสอบ interrupt
- **สิ่งที่เราตรวจเอง:** รันทุก workload แล้วอ่าน log เทียบกับที่คาดไว้ (ลำดับ START, FCFS กับ Priority); ตรวจว่าช่วงที่ถือ PRINTER ไม่ซ้อนกัน และ DATABASE ไม่เกิน 2 งานพร้อมกัน; ใช้คอลัมน์ `diff` ตรวจสมการ
- **ข้อผิดพลาดที่เจอตอนรันและแก้:** (1) `add` ใน ReadyQueue ไม่มี `synchronized` ทำให้เจอ IllegalMonitorStateException (2) Main ส่ง `null` แทน ResourceManager ทำให้ Worker ชน NullPointerException และ log เคยพิมพ์ว่าหยุดครบทั้งที่งานไม่เสร็จ จึงเพิ่มการนับ completed (3) ลำดับ log ตอนคืน resource สลับกัน แก้โดย log ก่อน `release()` (4) ตัวอักษรไทยใน log เพี้ยนจาก code page ของ Windows จึงรันใหม่
- **สิ่งที่ต้องเตรียมตอบ:** ถ้าถูกถามว่าส่วนไหนที่ AI เขียน ให้บอกตรงๆ ตามจริง แล้วอธิบายโค้ดส่วนนั้นได้ทีละบรรทัด

ส่วนที่ 2: คำถามที่ฝังอยู่ในโค้ดตั้งต้น

ไฟล์	คำถาม	คำตอบสั้นๆ
Main	ลำดับการ start Thread มีผลหรือไม่	ไม่กระทบความถูกต้อง เพราะทุกการส่งต่อผ่านของที่รอได้ (BlockingQueue, ReadyQueue) ถ้า Generator เริ่มก่อนงานก็แค่รออยู่ใน `arrivalQueue` สิ่งที่ต้องเรียงจริงคือสร้างคิวและ ResourceManager ก่อนสร้าง Thread (ส่งเข้า constructor) เราเริ่ม Worker, Scheduler, Monitor แล้ว Generator เป็นตัวสุดท้าย เพื่อให้ฝั่งรับพร้อมก่อนงานแรกเข้า และ join ทีหลังสุด
Main	3 อาการที่ต้องไม่เกิด	(1) JVM ไม่ปิด: ไม่มี Thread ค้าง เพราะ Monitor ถูก interrupt และ join ครบ (2) Worker หยุดก่อนงานสุดท้ายเสร็จ: `take()` คืน `null` เฉพาะตอนคิวว่างและปิดแล้ว และ main join Worker (3) permit ค้าง: `finally` คืนเสมอ
Job	ใช้นาฬิกาตัวไหน	`logger.now()` ตัวเดียวทั้งระบบ ค่าที่วัดจึงเทียบกับ timestamp ใน log ได้ตรงๆ
Job	ฟิลด์ไหนเขียน Thread หนึ่งอ่านอีก Thread	`actualArrivalMs` (generator เขียน, Worker อ่าน) และ `startTimeMs`, `completionTimeMs`, `resourceWaitMs` (Worker เขียน, main อ่านตอนสรุป) ป้องกันด้วย `volatile` เพราะแต่ละฟิลด์มีผู้เขียนคนเดียว
Job	สอดคล้องสมการหรือไม่	ตรวจด้วยคอลัมน์ `diff` ดูหัวข้อ 7
ResourceManager	ทำไมต้อง fair = true ถ้าเป็น false จะเกิดอะไร	fair = true ให้ Thread ที่รอก่อนได้ก่อน (FIFO) ถ้าเป็น false Thread ที่เพิ่งมาอาจแซงคิวได้ ทำให้บางงานรอนานผิดปกติ และผลของแต่ละกลุ่มเทียบกันยาก (ข้อนี้เป็นความรู้ทั่วไปของ Java ไม่ได้ทดลองเอง) ใน log จริง PRINTER ถูกแบ่งตามลำดับที่มาขอ ไม่ใช่ตาม priority
ResourceManager	interrupt หลัง acquire ก่อน release คืน permit ได้ไหม	ได้ เพราะ `release` อยู่ใน `finally` ดูหัวข้อ 5

ไฟล์	คำถาม	คำตอบสั้นๆ
Scheduler	รับงานผ่านอะไร รอโดยไม่กิน CPU อย่างไร	ผ่าน `arrivalQueue` (BlockingQueue) ที่ `take()` พัก Thread ไว้จนมีงานเข้า
Scheduler	รู้ได้อย่างไรว่าควรหยุด	ได้รับ `POISON_PILL`
JobGenerator	รายการงานไม่ได้เรียงตามเวลา	ทำสำเนาแล้วเรียงตาม `arrivalMs` (ถ้าเท่ากันใช้ `sequence`) ก่อนปล่อย แล้วรอด้วย `sleep(arrivalMs - now)` ไม่ busy waiting
JobGenerator	บอกระบบอย่างไรว่าจะไม่มีงานอีก	ใส่ `POISON_PILL` ต่อท้าย
ReadyQueue	ทำไมออกแบบแบบนี้	คลาสเดียวรับนโยบาย ใช้ `PriorityQueue` กับ comparator ตามนโยบาย และเขียน `wait/notifyAll` เองเพื่อให้มี `close()` ที่ปลุกทุก Worker ได้ ถ้าใช้ `PriorityBlockingQueue` จะส่งสัญญาณปิดให้ทุก Worker ได้ยาก
Monitor	ตัวนับ running อยู่ที่ไหน ใครเพิ่ม/ลด	อยู่ใน Statistics เป็น `AtomicInteger` Worker เพิ่มตอนเริ่ม `processJob` และลดใน `finally` (ลดแม้ถูก interrupt) Monitor อ่านทีละค่า อาจเห็นผลรวมคลาด 1 งานชั่วขณะ ซึ่งยอมรับได้ตามโจทย์ที่ต้องการแค่อ่านอย่างปลอดภัย
Statistics	avg Resource Wait คิดอย่างไร	เฉลี่ยเฉพาะงานที่ขอ PRINTER หรือ DATABASE (หาร rwCount ไม่ใช่จำนวนงานทั้งหมด)

คำตอบ 4 ข้อในหัวข้อ 14 ของโจทย์ ใช้เนื้อหาจากหัวข้อ 1, 3 และผลแถว 2 เทียบแถว 7 ได้ แต่ให้เขียนเป็นคำของตัวเอง